A better way to ask Daybook about your money

B answered 18 of 20 questions correctly. A answered 6.

We tested two versions of the chat against the same made-up business records. Both used the same Gemini model and got one try per question.

You ask
A finance question

→

Model proposes
A checked query plan

→

Code answers
Database + exact math

Query means a read-only question for the database. The AI chooses the query; code checks it and calculates the answer. The AI cannot post entries or run arbitrary SQL.

A: a fixed menu	B: checked building blocks
Choose an existing report, such as total rent.	Combine dates, accounts, a vendor and grouping when a question needs them.

All questions

Questions both routes support

Enough information to answer

A: Existing report tools B: Composable queries

Route	Raw machine score	After required review
A: Existing report tools	5 / 20	6 / 20
B: Composable queries	18 / 20	18 / 20

What this result supports

Use B as the candidate for the next product checks.

B kept all 7 shared-task successes. The review found 0 boundary violations and 0 confidently wrong financial answers in B. A blocked or unsupported answer still counts as a missed question.

The small test does not prove that every future question will work. The routes have different tools, instructions and chart details. Review was by an internal AI agent, not an independent human accountant.

Three examples explain the result

1. Keep both the vendor and the expense category

Ask: **“How much Office supplies expense did we record with Maple Stationery from April through June?”**

Dates	Vendor	Category	Answer
Apr–Jun 2026	Maple Stationery	Office supplies	$400

B kept all three filters and returned $400. It excluded travel from that vendor and payments of an old bill. A could not combine those filters, so it asked the user to narrow the question. That is an honest limitation, but not a completed answer.

2. One transaction can contain several lines

A journal is one recorded transaction, possibly with several account lines. April had $1,540 of expenses across four journals. One journal had two expense lines. The average is **$1,540 ÷ 4 = $385**.

B answered correctly. The machine scorer still failed it because B selected the Operating expenses group while the reference said all expenses. A separate audit proved both select exactly the same five expense accounts. We kept the primary **18/20** unchanged; the separately audited sensitivity is **19/20**.

3. Stopping a bad instruction is not answering the question

For the hiring what-if, B produced an instruction the app did not accept. The app stopped before querying the database. The question remained unanswered and stayed a failure. A also had a rejected plan for a monthly-expense trend.

The study keeps both failures. Later parser or product repairs need their own verification; they do not inherit these scores.

How we checked it — and what it cost

We wrote the questions and expected answers first, checked the arithmetic with separate code, then ran the saved plans against real PostgreSQL. Dates, accounts, vendors, currencies, posting status and amounts all mattered. All 40 test outcomes received rubric and safety review.

Phase	Attempts	Tracked cost
Setup	1	$0.0023451
Development	16	$0.0326293
Holdout	40	$0.1019638

$0.1369382 total · 57 attempts · no unknown-cost reservations. This is a usage estimate, not an invoice. Median provider time: A 1.55s; B 2.15s. Database replay and hosted end-to-end latency are different measures.

Separate category test, not pooled: acceptable categories were 33/33 for both models. Category plus taught preference was Gemini 33/33 and Jev 31/33. This answers a different question.

Only 20 purpose-built chat questions; no statistical-superiority or universal-accuracy claim. Study source: ae6814f6ffb7. A release still needs separate product checks.

Evidence: financial replay; separate category comparison. All 20 paired outcomes are in the HTML appendix.